Российский государственный университет нефти и газа (НИУ)
имени И.М. Губкина

ОТЧЕТ
о выполнении домашнего задания №5
по дисциплине «Методы оптимизации»

Выполнил: Новикова Дарья Дмитриевна
Проверил: Рыжова Лейла Лемаевна
Группа: КА-22-06

Москва, 2024

Постановка задачи

1. Для каждого варианта решить исходную задачу из предыдущего домашнего задания симплекс-методом:
   а. Найти начальную угловую точку (1 этап симплекс-метода). Для этого **предварительно подготовить задачу**, добившись неотрицательности вектора правой части системы b. Затем применить **метод искусственного базиса** и **решить вспомогательную задачу** вида:

$$\max L_1 = -\sum_{i=1}^{m} y_i$$

$$D: x| \begin{cases} y + Ax = b \\ L - c^T x = 0 \\ x \geq 0 \\ y \geq 0 \end{cases}$$

   выбирая здесь разрешающий элемент по критериям симплекс-метода и просматривая при выборе разрешающего столбца коэффициенты строки L_1. На каждой итерации необходимо удалять столбцы y_i после вывода их из базиса (так же, как и делали).

   б. Если L_1^{opt}=0, удалить строку L_1. Имеете начальную угловую точку и начальную симплекс-таблицу для нее.

   в. Найти оптимальный опорный план (2 этап симплекс-метода).

Для реализации метода программно реализовать алгоритм для метода перебора – написав или модифицировав уже имеющуюся процедуру. Здесь ввод и вывод переменных будет осуществляться уже на основе критериев симплекс-метода. Фиксировать все промежуточные таблицы с опорными решениями и соответствующим для них значением функции цели.

2. (*Отметить на графике геометрического метода начальную угловую точку и все остальные, которые были пройдены в ходе реализации симплекс-метода.*) Рядом с каждой таблицей, дающей опорное решение, написать имя соответствующей ей угловой точки из графического решения.

Исходные данные

Вариант 12.

c= -1 3 -8 6 -2

A=
6 4 1 8 8
-7 4 -1 0 7
9 6 7 3 1

b= 27 3 26

Решение

Запишем коэффициенты системы в таблицу, введя дополнительную строку для функции L1, выраженную предварительно через небазисные переменные. Можно проверить, что коэффициенты в ней при небазисных переменных будут равны сумме с обратным знаком соответствующих коэффициентов в строках, содержащих переменные yi в качестве базисных

+	x1	x2	x3	x4	x5	=
y1	6	4	1	8	8	27
y2	-7	4	-1	0	7	3
y3	9	6	7	3	1	26
L	1	-3	8	-6	2	0
L1	-8	-14	-7	-11	-16	-56

I этап. Поиск начального опорного решения: L1—> max.

+	x1	x2	x3	x4	x5	=
y1	6	4	1	8	8	27
y2	-7	4	-1	0	7	3
y3	9	6	7	3	1	26
L	1	-3	8	-6	2	0
L1	-8	-14	-7	-11	-16	-56

+	x1	x2	x3	x4	=
y1	14	-4/7	2 1/7	8	23 4/7
x5	-1	4/7	-1/7	0	3/7
y3	10	5 3/7	7 1/7	3	25 4/7
L	3	-4 1/7	8 2/7	-6	-6/7
L1	-24	-4 6/7	-9 2/7	-11	-49 1/7

+	x1	x2	x3	x4	=
y1	14	-4/7	2 1/7	8	23 4/7
x5	-1	4/7	-1/7	0	3/7
y3	10	5 3/7	7 1/7	3	25 4/7
L	3	-4 1/7	8 2/7	-6	-6/7
L1	-24	-4 6/7	-9 2/7	-11	-49 1/7

+	x2	x3	x4	=
x1	- 2/49	15/98	4/7	1 67/98
x5	26/49	1/98	4/7	2 11/98
y3	5 41/49	5 30/49	-2 5/7	8 36/49
L	-4 1/49	7 81/98	-7 5/7	-5 89/98
L1	-5 41/49	-5 30/49	2 5/7	-8 36/49

+	x2	x3	x4	=
x1	- 2/49	15/98	4/7	1 67/98
x5	26/49	1/98	4/7	2 11/98
y3	5 41/49	5 30/49	-2 5/7	8 36/49
L	-4 1/49	7 81/98	-7 5/7	-5 89/98
L1	-5 41/49	-5 30/49	2 5/7	-8 36/49

+	y3	x3	x4	=
x1	1/143	5/26	79/143	1 213/286
x5	- 1/11	- 1/2	9/11	1 7/22
x2	49/286	25/26	-133/286	1 71/143
L	197/286	11 9/13	-9 167/286	31/286
L1	1	0	0	0

Получено первое опорное решение: $x_1 = \frac{499}{286}$, $x_2 = \frac{214}{143}$, $x_5 = \frac{29}{22}$

Значение функции цели на нем равно $L = \frac{31}{286} \approx 0{,}1084$

Это точка E на графике ниже.

II этап. Поиск оптимального опорного решения: L —> max.

+	x3	x4	=
x1	5/26	79/143	1 213/286
x5	- 1/2	9/11	1 7/22
x2	25/26	-133/286	1 71/143
L	11 9/13	-9 167/286	31/286

+	x3	x4	=
x1	5/26	79/143	1 213/286
x5	- 1/2	9/11	1 7/22
x2	25/26	-133/286	1 71/143
L	11 9/13	-9 167/286	31/286

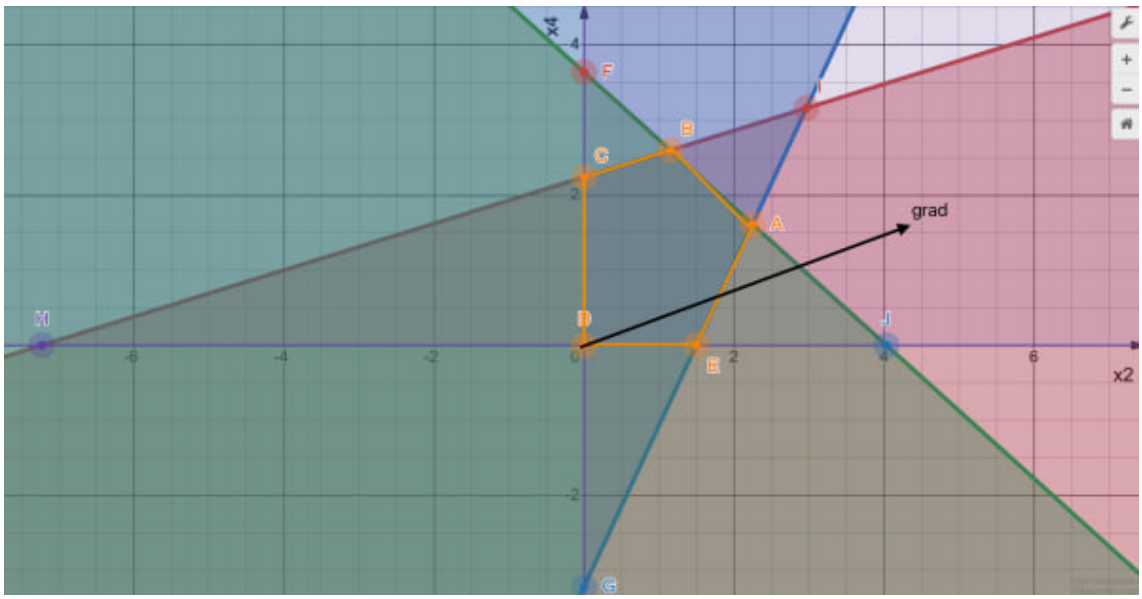
+	x3	x5	=
x1	62/117	- 79/117	100/117
x4	- 11/18	1 2/9	1 11/18
x2	317/468	133/234	2 115/468
L	5 391/468	11 167/234	15 257/468

Коэффициенты при небазисных переменных в L-строке неотрицательны, значит найдено оптимальное решение. Координаты этого оптимального решения: $x_1 = \frac{100}{117}$, $x_2 = \frac{1051}{468}$, $x_4 = \frac{29}{18}$

Значение функции цели на нем равно $L = \frac{7277}{468} \approx 15{,}5492$

На графике это точка А.

График геометрического метода



Программная реализация

Принцип работы программы:

1. Создается начальная таблица из заданных коэффициентов системы ограничений и целевых функций. Все данные представляются в виде дробей для точности вычислений, а также простоты сравнения с результатами вручную.
2. В процессе работы алгоритм выбирает разрешающий элемент для выполнения операции симплекс-метода.
3. Начинает решение задачи с целевой функции L1, затем, после того как все коэффициенты в строке L1 станут неотрицательными, переходит к целевой функции L. Для каждой целевой функции выполняется цикл улучшений, пока не будет найдено оптимальное решение. После каждого шага выводит таблицу с результатми.
4. В конце алгоритм выводит: координаты оптимальной точки, значение целевой функции L в оптимальной точке.

В программе используется несколько функций, каждая из которых выполняет определенную задачу. Это сделано для того, чтобы можно было легко отслеживать и контролировать процесс решения, а также повысить читаемость программы.

Если за время работы алгоритма был выполнен один и тот же шаг несколько раз, программа обнаруживает это и завершает выполнение с ошибкой. Также предусмотрена проверка если решение невозможно из-за отсутствия положительных разрешающих строк.

Скриншоты результата работы программы:

```
Исходная таблица:
        x1      x2      x3      x4      x5      =
y1      6       4       1       8       8       27
y2      -7      4       -1      0       7       3
y3      9       6       7       3       1       26
L       1       -3      8       -6      2       0
L1      -8      -14     -7      -11     -16     -56
Шаг 0 для L1:
        x1      x2      x3      x4      x5      =
y1      6       4       1       8       8       27
y2      -7      4       -1      0       7       3
y3      9       6       7       3       1       26
L       1       -3      8       -6      2       0
L1      -8      -14     -7      -11     -16     -56
Выбран разрешающий столбец для L1: x5
Выбрана разрешающая строка для L1: y2
Шаг 1 для L1:
        x1      x2      x3      x4      x5      =
y1      14      -4/7    15/7    8       0       165/7
x5      -1      4/7     -1/7    0       1       3/7
y3      10      38/7    50/7    3       0       179/7
L       3       -29/7   58/7    -6      0       -6/7
L1      -24     -34/7   -65/7   -11     0       -344/7
Выбран разрешающий столбец для L1: x1
Выбрана разрешающая строка для L1: y1
Шаг 2 для L1:
        x1      x2      x3      x4      x5      =
x1      1       -2/49   15/98   4/7     0       165/98
x5      0       26/49   1/98    4/7     1       207/98
y3      0       286/49  275/49  -19/7   0       428/49
L       0       -197/49 767/98  -54/7   0       -579/98
L1      0       -286/49 -275/49 19/7    0       -428/49
Выбран разрешающий столбец для L1: x2
Выбрана разрешающая строка для L1: y3
```

```
Шаг 3 для L:
        x1      x2      x3      x4      x5      =
x1      1       0       5/26    79/143  0       499/286
x5      0       0       -1/2    9/11    1       29/22
x2      0       1       25/26   -133/286        0       214/143
L       0       0       152/13  -2741/286       0       31/286
L1      0       0       0       0       0       0
Выбран разрешающий столбец для L: x4
Выбрана разрешающая строка для L: x5
Финальная таблица:
        x1      x2      x3      x4      x5      =
x1      1       0       62/117  0       -79/117 100/117
x4      0       0       -11/18  1       11/9    29/18
x2      0       1       317/468 0       133/234 1051/468
L       0       0       2731/468        0       2741/234        7277/468
L1      0       0       0       0       0       0
Оптимальная точка:
x1 = 0.85
x4 = 1.61
x2 = 2.25
Значение функции L в оптимальной точке: 15.55
```

Выводы

Была решена задача линейного программирования симплекс-методом вручную и с помощью программной реализации. В результате была найдена оптимальная точка и значение целевой функции в ней. А также можно заметить, что симплекс-метод является более эффективным методом для решения задач линейного программирования по сравнению с методом полного перебора.

Приложения

Листинг кода:

```
import numpy as np

from fractions import fraction


def to_fractions(data):

    # преобразует данные в дроби с наименьшими знаменателями

    return np.array([[fraction(x).limit_denominator() for x in row] for row in data])



def initialize_tableau():

    # создает начальную таблицу для симплекс-метода

    data = [

        [6, 4, 1, 8, 8, 27],

        [-7, 4, -1, 0, 7, 3],

        [9, 6, 7, 3, 1, 26],

        [1, -3, 8, -6, 2, 0],  # целевая функция l

        [-8, -14, -7, -11, -16, -56]  # целевая функция l1

    ]

    index = ['y1', 'y2', 'y3', 'l', 'l1']

    columns = ['x1', 'x2', 'x3', 'x4', 'x5', '=']

    return index, columns, to_fractions(data)
```

```
def print_table(index, columns, tableau):
    # выводит текущую таблицу симплекс-метода в читаемом виде
    header = "\t".join([""] + columns)
    print(header)
    for idx, row in zip(index, tableau):
        print("\t".join([idx] + [str(x) for x in row]))



def perform_pivot(index, columns, tableau, razr_row, razr_col):
    # выполняет операцию разрешающего элемента для симплекс-метода
    razr_value = tableau[razr_row, razr_col]

    # нормализуем разрешающую строку, делим ее на разрешающий элемент
    tableau[razr_row] = tableau[razr_row] / razr_value

    # обновляем остальные строки таблицы с учетом новой разрешающей строки
    for i in range(len(tableau)):
        if i != razr_row:  # не изменяем разрешающую строку
            factor = tableau[i, razr_col]
            tableau[i] -= factor * tableau[razr_row]
```

```
    return index, columns, tableau



def apply_simplex():
    # основной процесс решения задачи с помощью симплекс-метода
    index, columns, tableau = initialize_tableau()
    print("исходная таблица:")
    print_table(index, columns, tableau)

    iteration = 0
    previous_pivots = set()  # защита от зацикливания

    # начинаем работать сначала с l1, а потом с l
    while true:
        # проверяем строку l1 на наличие отрицательных значений
        if any(x < 0 for x in tableau[-1, :-1]):
            print(f"шаг {iteration} для l1:")
            print_table(index, columns, tableau)

            # находим разрешающий столбец для l1 (столбец с минимальным значением)
            razr_col = np.argmin(tableau[-1, :-1])
            print(f"выбран разрешающий столбец для l1: {columns[razr_col]}")
```

```
# находим разрешающую строку
ratios = []
for i in range(len(tableau) - 2):  # исключаем строки l и l1
    if tableau[i, razr_col] > 0:
        ratios.append((tableau[i, -1] / tableau[i, razr_col], i))
    else:
        ratios.append((float('inf'), i))

if all(r[0] == float('inf') for r in ratios):  # если все отношения бесконечны
    raise exception("задача неограничена.")

# выбираем разрешающую строку с минимальным отношением
razr_row = min(ratios, key=lambda x: x[0])[1]
print(f"выбрана разрешающая строка для l1: {index[razr_row]}")

# проверка на зацикливание
if (razr_row, razr_col) in previous_pivots:
    print(f"такая позиция уже использовалась: ({index[razr_row]}, {columns[razr_col]})")
    raise exception("симплекс-метод зациклился. проверьте постановку задачи.")
previous_pivots.add((razr_row, razr_col))
```

```
        # выполняем операцию разрешающего элемента
        index, columns, tableau = perform_pivot(index, columns, tableau, razr_row, razr_col)

        # обновляем базисную переменную (название строки)
        if index[razr_row] not in ['l', 'l1']:
            index[razr_row] = columns[razr_col]

        iteration += 1
    else:
        # если с l1 все завершено, начинаем работу с l
        print(f"шаг {iteration} для l:")
        print_table(index, columns, tableau)

        # находим разрешающий столбец для l (столбец с минимальным значением)
        razr_col = np.argmin(tableau[-2, :-1])
        print(f"выбран разрешающий столбец для l: {columns[razr_col]}")

        # находим разрешающую строку
        ratios = []
        for i in range(len(tableau) - 2):  # исключаем строки l и l1
```

```
            if tableau[i, razr_col] > 0:
                ratios.append((tableau[i, -1] / tableau[i, razr_col], i))
            else:
                ratios.append((float('inf'), i))

        if all(r[0] == float('inf') for r in ratios):  # если все отношения бесконечны
            raise exception("задача неограничена.")

        # выбираем разрешающую строку с минимальным отношением
        razr_row = min(ratios, key=lambda x: x[0])[1]
        print(f"выбрана разрешающая строка для l: {index[razr_row]}")

        # проверка на зацикливание
        if (razr_row, razr_col) in previous_pivots:
            print(f"такая позиция уже использовалась: ({index[razr_row]}, {columns[razr_col]})")
            raise exception("симплекс-метод зациклился. проверьте постановку задачи.")
        previous_pivots.add((razr_row, razr_col))

        # выполняем операцию разрешающего элемента для l
        index, columns, tableau = perform_pivot(index, columns, tableau, razr_row, razr_col)
```

```
        # обновляем базисную переменную (название строки)
        if index[razr_row] not in ['l', 'l1']:
            index[razr_row] = columns[razr_col]

        iteration += 1

    # проверяем завершение работы для обеих целевых функций
    if all(x >= 0 for x in tableau[-1, :-1]) and all(x >= 0 for x in tableau[-2, :-1]):
        print("финальная таблица:")
        print_table(index, columns, tableau)
        break

# находим оптимальное решение
optimal_values = {}
for i, row_name in enumerate(index[:-2]):  # исключаем строки l и l1
    if row_name not in ['l', 'l1']:
        column_index = columns.index(row_name)
        optimal_values[columns[column_index]] = tableau[i, -1]

# выводим оптимальные координаты
print("оптимальная точка:")
for var, value in optimal_values.items():
```

```
        print(f"{var} = {float(value):.2f}")


    # вычисляем значение целевой функции
    optimal_value = tableau[-2, -1]
    print(f"значение функции l в оптимальной точке: {float(optimal_value):.2f}")
# вызов функции для выполнения симплекс-метода
apply_simplex()
```